

September 14, 2026

Flare Relay Network Diff Smart Contract Patch Review

Placeholder content for the Smart Contract Patch Review report.

Contents

About Zellic	4
1. Introduction	4
1.1. Scope	5
1.2. Disclaimer	6
2. Detailed Findings	6
2.1. Merkle verification delegated to the old relay accepts a zero leaf for unfinalized rounds	7
2.2. Governance-selected exact call value relaxes the timelock guarantee	9
2.3. Protocol message events can contain a noncanonical boolean	11
2.4. The arbitrary-calldata self-call in <code>_verifyCustomSignature</code> could become dangerous	13
2.5. A blank <code>_authorizeUpgrade</code> implementation could become dangerous	15
2.6. Ownership transfers are not subject to the timelock	17
2.7. For larger signer sets, mandating that the signatures be sorted by signer can save gas	18
2.8. The <code>noSigningPolicyRelay</code> flag should be used in places where <code>signingPolicySetter</code> is checked instead	20
3. Discussion	21
3.1. The Relay initialization process can check the deployment chain ID in both branches	22
3.2. The <code>TimelockDurationSet</code> event logic could be a helper function	22

4.	Patch Review	24
4.1.	Notable changes	25
4.2.	Minor changes	26

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Introduction

We were asked to review a patch to the Flare Relay network from August 25th to September 1st, 2026 for a total of 1.5 person-weeks.

1.1. Scope

The engagement involved a review of the following targets:

Flare Relay Network Diff Contracts

Type	Solidity
Platform	EVM-compatible
Target	Only changes between e75cabb...13c8654
Repository	https://github.com/flare-foundation/flare-smart-contracts-v2 ↗
Version	13c8654014ef0f852cb3f3b1db162f9669a605ce
Programs	contracts/protocol/implementation/Relay.sol contracts/protocol/implementation/RelayProxy.sol contracts/utils/implementation/OwnableWithTimelock.sol contracts/utils/implementation/Create3Factory.sol contracts/fdc2/implementation/Fdc2Verification.sol contracts/fdc2/library/Fdc2ProofVerification.sol contracts/protocol/implementation/FtsoV2.sol contracts/customFeeds/implementation/SFlrCustomFeed.sol contracts/fscV1/implementation/FtsoProxy.sol contracts/fscV1/implementation/PriceSubmitterProxy.sol contracts/governance/implementation/FlareGovernedBase.sol contracts/governance/lib/FlareGovernance.sol contracts/tee/facets/MachineManagerFacet.sol contracts/tee/library/ExtensionGovernance.sol

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✧ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Weipeng Lai
✧ Security Engineer
weipeng.lai@zellic.io ↗

Kuilin Li
✧ Security Engineer
kuilin@zellic.io ↗

1.2. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.

2. Detailed Findings

2.1. Merkle verification delegated to the old relay accepts a zero leaf for unfinalized rounds

Target	Relay		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

This diff hardened the local branch of the `verify()` function in Relay so that a voting round without a finalized root can no longer serve as a verification target. The stored root is read first and rejected when it is zero:

```
bytes32 root = merkleRootsPrivate[_protocolId][_votingRoundId];
require(root != bytes32(0), NotFinalized());
require(_proof.verifyCalldata(root, _leaf), MerkleProofInvalid());
```

The guard is necessary because `MerkleProof.processProofCalldata` returns the leaf unchanged when the proof is empty, which reduces `verifyCalldata` to an equality test between the leaf and the root. A caller supplying a zero leaf and an empty proof therefore matches the zero root that every unfinalized round holds.

However, the branch taken before it, which serves rounds below the migration boundary out of the previously deployed relay, carries no equivalent guard. It forwards the caller's arguments unchanged and only requires that the delegated verification call return true:

```
if (address(oldRelay) != address(0) && _votingRoundId <
    startingVotingRoundIdForInitialRewardEpochId) {
    // [...]
    bool ok = oldRelay.verify(_protocolId, _votingRoundId, _leaf, _proof);
    require(ok, OldRelayVerificationFailed());
    // [...]
    return true;
}
```

The legacy relay implementation does not enforce a finalization check of its own. It verifies proofs directly against the stored root without checking that it is nonzero. Consequently, a caller can select an unfinalized round below the migration boundary, supply `_leaf = bytes32(0)` and an empty `_proof`, and obtain true from `verify()`.

Impact

For voting round below the migration boundary that holds no root, the Relay can report a successful verification for a zero leaf that no signing policy ever committed to, so the finalization guarantee the interface documents does not hold on historical rounds.

The demonstrated bypass is limited to a zero leaf with an empty proof against a zero root. It does not establish arbitrary nonzero leaf forgery. Consumers that derive their leaf as the hash of an encoded payload are therefore unaffected in practice.

Recommendations

We recommend gating the delegated branch on the old relay's own finalization view before forwarding the proof, which mirrors the invariant the local branch enforces:

```
require(oldRelay.isFinalized(_protocolId, _votingRoundId), NotFinalized());  
bool ok = oldRelay.verify(_protocolId, _votingRoundId, _leaf, _proof);  
require(ok, OldRelayVerificationFailed());
```

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit [53056a45](#) [n](#).

2.2. Governance-selected exact call value relaxes the timelock guarantee

Target	FlareGovernedBase, FlareGovernance		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

In order to support the payment of native value to governance-call targets, this diff added the payable modifier to the executeGovernanceCall function, implemented in the governance timelock abstract contract. At execution time, this forwards the given value to the call:

```
/**
 * Execute a timelocked governance call.
 * @param _encodedCall The full encoded call data (hash verified against
 * stored hash).
 */
// False positive: slither sees the library alone, not its one caller,
// `FlareGovernedBase.executeGovernanceCall`, which is `external payable`.
//slither-disable-next-line msg-value-in-nonpayable
function executeGovernanceCall(
    bytes calldata _encodedCall
)
    internal
{
    // [...]
    state.executing = true;
    // Forward any attached value so payable governance methods can receive
    // funds at
    // execution time; a non-payable target rejects it, reverting the
    // execution.
    //solhint-disable-next-line avoid-low-level-calls
    (bool success,) = address(this).call(_encodedCall);
    (bool success,) = address(this).call{value: msg.value}(_encodedCall);
    state.executing = false;
    emit IFlareGovernance.TimelockedGovernanceCallExecuted(callHash);
    _passReturnOrRevert(success);
}
```

This means that the amount of native value sent with the call is decided at execution time, instead of when the call is timelocked.

Impact

This relaxes the timelock guarantee a little. The amount of native value to send through the time-locked call can change what the call does, especially if the call target branches on the amount of native value it receives.

The current impact is minimal, because the only timelocked functions that take native value are ones that use it on a fee. For these, the fee is either paid or unpaid, and so the timelock itself does meaningfully commit the contract to a specific action.

Recommendations

There is no clean way to resolve this, because of two considerations. Currently, there is a UX convenience where governance calls that become timelocked can be issued by calling the function normally, rather than having to encode the calldata and then call a general function. The second consideration is that the contract should not, in general, store native value between transactions. If both considerations are kept, then committing to an amount of native value for the duration of the timelock would require storing it in a separate contract maintained by the FlareGovernance abstract contract.

If additional payable governance timelocked functions are planned, then we recommend revisiting this design choice to ensure that end users still benefit from the same degree of timelock guarantees.

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit [cbd3b8e1](#). This commit documents the behavior, clarifying that the choice of value escapes being timelocked.

2.3. Protocol message events can contain a noncanonical boolean

Target	Relay		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

The relay function parses `isSecureRandom` as a byte from the packed protocol message. For protocol IDs greater than 1, it does not reject values outside `{0, 1}`. The signed digest commits to the raw message bytes, so a message containing a noncanonical value can be accepted if the required signing quorum signs those exact bytes and the other relay checks pass.

The `randomNumberProtocolId` path normalizes the byte using `iszero(iszero(...))` before using it in the random-number proof, storage, and events. However, the other protocol-finalization path copies the masked byte directly into event data:

```
mstore(add(memPtrFor, M_5_isSecureRandom),
  structValue(
    mload(memPtrFor),
    MSG_NMR_BOFF_isSecureRandom,
    MSG_NMR_MASK_isSecureRandom
  )
)
mstore(add(memPtrFor, M_3), "ProtocolMessageRelayed(uint8,uin")
mstore(add(memPtrFor, M_4), "t32,bool,bytes32)")
log3(
  add(memPtrFor, M_5_isSecureRandom), 64, keccak256(add(memPtrFor, M_3), 49),
  protocolId, votingRoundId
)
```

The mask `MSG_NMR_MASK_isSecureRandom` is `0xff`, so it preserves values such as 2. Although `ProtocolMessageRelayed` declares `isSecureRandom` as `bool`, the assembly `log3` bypasses Solidity's event encoder and emits the value unchanged. This violates the ABI encoding requirement for booleans, which permits only 0 and 1.

Impact

If a quorum signs a message with an `isSecureRandom` byte above 1 for a protocol other than `randomNumberProtocolId`, the relay can finalize the Merkle root while emitting a log that strict ABI decoders reject. An indexer relying on decoded `ProtocolMessageRelayed` events may therefore fail to

process that finalization or interrupt log ingestion, depending on its error handling.

Recommendations

We recommend rejecting any `isSecureRandom` byte above 1 during protocol-message parsing.

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit [e583bb72](#).

2.4. The arbitrary-calldata self-call in _verifyCustomSignature could become dangerous

Target	Relay		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `verifyCustomSignature` and `verifyCustomSignatureWithThreshold` functions have a `_relayMessage` argument which is entirely caller-controlled. These functions then call `_verifyCustomSignature` which does a self-call with that calldata:

```
function _verifyCustomSignature(
    bytes calldata _relayMessage,
    bytes32 _messageHash
) internal returns (uint256 _rewardEpochId) {
    /* solhint-disable avoid-low-level-calls */
    //slither-disable-next-line arbitrary-send-eth
    (bool success, bytes memory returnData)
    = address(this).call(_relayMessage);
    /* solhint-enable avoid-low-level-calls */
    require(success, VerificationFailed());
    // The 35-byte return length (32-byte hash plus 3-byte reward epoch id)
    uniquely identifies
    // relay()'s protocolId == 1 path; every other path returns no data or
    reverts. Changes to
    // relay() return formats must preserve this discriminator or replace it
    with a typed one.
    require(returnData.length == 35, WrongVerificationData());
    bytes32 returnHash;
    uint256 returnRewardEpochId;
    // solhint-disable-next-line no-inline-assembly
    assembly {
        returnHash := mload(add(returnData, 0x20))
        returnRewardEpochId := shr(sub(256, mul(8, REWARD_EPOCH_ID_BYTES)),
        mload(add(returnData, 0x40)))
    }
    require(bytes32(returnHash) == _messageHash, InvalidConfigHash());
    return returnRewardEpochId;
}
```

This workflow expects `_relayMessage` to encode a call to `relay()`, but, the caller can encode any call there. It is also unpermissioned, so anyone can be the caller. The only guards here are that the return data must be 35 bytes long, and a subsequence of that data matches `_messageHash`, which is also completely controlled by the caller.

Because the sender is the contract address itself for the nested call, this means that any function on this contract that happens to return 35 bytes can be made by anyone.

Impact

There is no current impact, because the only function that returns 35 bytes is the intended `relay()` call with the protocol ID set to 1.

However, there is no guard against a future update to this contract adding such a function. Having the contract call itself may expose privileges that unpermissioned callers lack, for instance, currently, `OwnableWithTimelock` has a self-call check in `executeTimelockedCall` which, if not for the `state.executing` check, would be exploitable.

Recommendations

We recommend checking the first four bytes of `_relayMessage` to ensure that it is actually a call to `relay()`.

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit [4ff9c6e0](#).

2.5. A blank `_authorizeUpgrade` implementation could become dangerous

Target	Relay		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

This diff makes the Relay contract upgradeable, adding the RelayProxy contract that is an ERC1967Proxy with its implementation set to the Relay contract. As a part of making it upgradeable, it also adds overrides for `upgradeToAndCall` and `_authorizeUpgrade` to implement authorization for the upgrades:

```
/**
 * Upgrades the UUPS implementation through the owner-timelock path: with a
 * nonzero
 * duration the exact call is queued for permissionless execution after its
 * ETA; with
 * zero it executes immediately. Only the per-chain owner can queue; on Flare
 * this is
 * Flare governance, on other chains the designated multisig.
 * @param _newImplementation The new implementation address.
 * @param _data Optional post-upgrade initialization calldata.
 */
function upgradeToAndCall(
    address _newImplementation,
    bytes memory _data
)
    public payable override
    onlyOwnerWithTimelock
{
    super.upgradeToAndCall(_newImplementation, _data);
}

/// @dev Empty: authorization is `onlyOwnerWithTimelock` on the
/// `upgradeToAndCall`
/// wrapper above.
/// solhint-disable-next-line no-empty-blocks
function _authorizeUpgrade(
    address _newImplementation
)
    internal override
```

```
{ }
```

The blank `_authorizeUpgrade` would allow anyone to upgrade the contract to any implementation, but it is only reachable through the `upgradeToAndCall` function, which has the timelock modifier that restricts it to governance.

Impact

There is no current impact. This contract pins the version of the OpenZeppelin dependency at 5.7.0, which only has one external upgrade function, the correctly-overridden `upgradeToAndCall`. However, since OpenZeppelin expects `_authorizeUpgrade` to be the function that actually authorizes the upgrade, any future version may add other functions or change other behavior that interacts poorly with this implementation.

Recommendations

We recommend checking that the sender is `address(this)` or that the calldata signature is `upgradeToAndCall` inside the `_authorizeUpgrade`, so that any future changes cannot cause this empty authorization check to allow anyone to upgrade this contract.

Remediation

This issue has been acknowledged by Flare.

2.6. Ownership transfers are not subject to the timelock

Target	Relay		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Relay inherits OwnableWithTimelock, which applies the configured delay only to functions using the onlyOwnerWithTimelock modifier. However, OwnableWithTimelock inherits transferOwnership from OpenZeppelin's OwnableUpgradeable without overriding it. The inherited implementation uses onlyOwner and updates ownership directly:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    if (newOwner == address(0)) {
        revert OwnableInvalidOwner(address(0));
    }
    _transferOwnership(newOwner);
}
```

Consequently, a successful call to transferOwnership takes effect immediately, even when timelockDurationSeconds is nonzero. Ownership transfers cannot be queued through the contract's built-in timelock.

Impact

The configured timelock provides no review or cancellation window for ownership changes. An incorrect transfer therefore takes effect immediately and cannot be canceled by the previous owner. This is an informational limitation of the governance design: transfers remain restricted to the owner, and the new owner remains subject to the configured delay for timelocked operations.

Recommendations

If ownership changes are intended to receive the same delay as other administrative operations, override transferOwnership to use onlyOwnerWithTimelock.

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit [8367d2d4](#).

- 2.7. For larger signer sets, mandating that the signatures be sorted by signer can save gas

Target	Fdc2Verification		
Category	Optimization	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

This diff modifies the `verifyTeeSignatures` process by adding an extension ID field, and abstracting out the verification function into a new `_verifyTeeSignatures`:

```
function _verifyTeeSignatures(
    IFlareTeeManager _flareTeeManager,
    uint256 _extensionId,
    Signature[] calldata _signatures,
    bytes32 _messageHash
)
    internal view
    returns (address[] memory _signingTeeIds)
{
    _signingTeeIds = new address[](_signatures.length);
    for (uint256 i = 0; i < _signatures.length; i++) {
        address teeId = _verifyTeeSignature(_flareTeeManager, _extensionId,
            _signatures[i], _messageHash);
        for (uint256 j = 0; j < i; j++) {
            require(_signingTeeIds[j] != teeId, DuplicatedTeeId(teeId));
        }
        _signingTeeIds[i] = teeId;
    }
}
```

The implementation of this function is quadratic in the number of signatures, because each signature needs to be checked for duplicates against each other signature.

Impact

This check does not need to be quadratic in gas cost. If there are many signers, then considerable gas can be saved by redesigning it.

Recommendations

If the interface can be changed, then we recommend requiring that the signatures be sorted by signer. This would easily make the signatures checkable in linear time, as each signature just needs to be compared against the next.

Remediation

Flare has acknowledged this issue and provided the following comment:

Since any change would require off-chain components to adopt it, and we expect at most 3–5 TEE machines and 3–5 cosigners, we have decided to leave it as is.

2.8. The noSigningPolicyRelay flag should be used in places where signing-PolicySetter is checked instead

Target	Relay		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

With the changes in the diff, the Relay contract can be deployed both in relay mode and in home mode. The noSigningPolicyRelay boolean flag is set depending on which mode it is, and the signingPolicySetter state is only nonzero if it is in home mode:

```
function initialize(
    RelayInitialConfig memory _initialConfig,
    address _signingPolicySetter,
    IRelay _oldRelay,
    address _initialOwner
)
external initializer
{
    // [...]
    if (_signingPolicySetter != address(0)) {
        // [...]
        signingPolicySetter = _signingPolicySetter;
        stateData.noSigningPolicyRelay = true;
        // [...]
    } else {
        // [...]
    }
    // [...]
}
```

The following exceptions check if signingPolicySetter is the zero address in order to branch based on the mode of the Relay contract:

- OldRelayNotAllowedInRelayMode() and OldRelayIncompatible(), only if noSigning-PolicyRelay is exposed externally
- FeeConfigNotAllowed()
- FeeExemptionsNotAllowed()
- FeeConfigNotAllowed()

- `SigningPolicySetterNotAllowed()`
- `NoAccessToMerkleRoots()`
- `NoAccessToSigningPolicyHashes()`

Impact

The source of truth for whether the Relay contract is in relay mode or home mode is the secondary variable `signingPolicySetter`, which relies on an invariant that the `signingPolicySetter` is nonzero if and only if it is in home mode. This makes it hard to change that invariant for future iterations of the code.

Recommendations

We recommend using the `noSigningPolicyRelay` flag instead of checking `signingPolicySetter` to check which mode the Relay contract is in.

Remediation

Flare has acknowledged this issue and provided the following comment:

Since the checks are functionally equivalent and the current one is marginally cheaper in gas, we have decided to leave the code unchanged.

3. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

3.1. The Relay initialization process can check the deployment chain ID in both branches

- code quality - in initialize, since the setter-mode branch checks `sourceChainId == block.chainid`, in the relay-mode branch, would it make sense to double-check that `sourceChainId` is not `block.chainid`?

With the changes in the diff, the Relay contract can be deployed both in relay mode and in home mode. The `noSigningPolicyRelay` boolean flag is set depending on which mode it is, and the `signingPolicySetter` state is only nonzero if it is in home mode:

The Relay contract can be initialized in either relay mode or home mode, depending on where the contract's deployment chain is on the hub-and-spoke multi-chain architecture. When the contract is initialized in home mode, the `sourceChainId` is checked to be equal to `block.chainid`:

```
if (_signingPolicySetter != address(0)) {
    // [...]
    // A home deployment must bind policy and message digests to its own chain.
    require(
        _initialConfig.sourceChainId == block.chainid,
        SourceChainIdMismatchOnHomeDeploy()
    );
    // [...]
} else {
    // [...]
}
```

There is no such check in the other branch, so it is possible that a relay-mode instance of the contract is deployed on the same chain as its `sourceChainId`. If this possibility is not intentional, then we recommend checking that the two are not equal in the other branch.

This issue has been acknowledged by Flare.

3.2. The `TimelockDurationSet` event logic could be a helper function

In the `initialize` function of the Relay contract, when the timelock duration is set, it checks it against `MAX_TIMELOCK_DURATION_SECONDS`, calls the `OwnableWithTimelock`-provided `getState()`

function to get a storage reference, and then emits an event:

```
function initialize(  
    RelayInitialConfig memory _initialConfig,  
    // [...]  
)  
    external initializer  
{  
    // [...]  
    require(  
        _initialConfig.timelockDurationSeconds  
        <= MAX_TIMELOCK_DURATION_SECONDS,  
        TimelockDurationTooLong()  
    );  
    getState().timelockDurationSeconds  
    = _initialConfig.timelockDurationSeconds;  
    emit TimelockDurationSet(_initialConfig.timelockDurationSeconds);  
}
```

This logic matches the logic in the OwnableWithTimelock implementation, which does it in the same way:

```
function setTimelockDuration(  
    uint256 _timelockDurationSeconds  
)  
    external  
    virtual  
    onlyOwnerWithTimelock  
{  
    State storage state = getState();  
    require(_timelockDurationSeconds <= MAX_TIMELOCK_DURATION_SECONDS,  
        TimelockDurationTooLong());  
    state.timelockDurationSeconds = _timelockDurationSeconds;  
    emit TimelockDurationSet(_timelockDurationSeconds);  
}
```

However, the details of this implementation should be owned by the OwnableWithTimelock contract, instead of being copy-pasted into the initialize function over at the Relay contract.

We recommend moving this implementation into a helper function in OwnableWithTimelock, so that the initialization logic can call it, instead of duplicating this logic.

Flare has acknowledged this issue and provided the following comment:

Since this contract is shared across multiple repositories and we want to remain aligned, we have decided not to change it.

4. Patch Review

The purpose of this section is to document the core changes in the codebase made within the commit range [e75cabba](#) to [13c86540](#).

4.1. Notable changes

The following are notable changes made to the codebase.

- **Source-chain binding for signatures and policies.** The headline change replaces iterative signing-policy hashing with `keccak256(sourceChainId | content)`: a single hash of the 32-byte source network ID concatenated with the raw encoded policy bytes. Signed protocol-message digests use the same construction with the raw message bytes, before the existing EIP-191 signature prefix is applied. This separates signatures and policy hashes belonging to different source networks even when their voter sets fully overlap. FDC2 cosigner hashing (`Fdc2ProofVerification.toCosignersMessageHash()`) adopts the same shape using `block.chainid`.
- **Upgradeable Relay, deterministic deployment, and owner administration.** Relay becomes a UUPS implementation, with initialization performed atomically by the new `RelayProxy` constructor. The proxy is deployed through the new permissionless `Create3Factory`, which derives deployment addresses from the factory, caller, and salt, independently of initialization code — supporting matching Relay proxy addresses across chains where the same factory address and deployer are used. Administration is per chain: the new `OwnableWithTimelock` contract governs upgrades, fee configuration, fee exemptions, fee collection, and signing-policy-setter changes. With a nonzero delay, the owner queues calls by their exact calldata and anyone can execute them after the delay; a zero delay permits immediate execution by the owner. Ownership transfer itself is not timelocked: `transferOwnership` is inherited unchanged from `OwnableUpgradeable` and takes effect immediately.
- **Merkle-proven random values.** A random-number-protocol message must now carry the random value and a Merkle proof after the signatures. The proven leaf binds the voting round, the value, and the `isSecureRandom` flag to the signed root, and the verified value is persisted per round and emitted in a new `RandomNumberRelayed` event.
- **Verification fees and exemptions.** On a mirror deployment, `Relay.verify()` charges a per-protocol fee in either the native coin or a configured ERC-20, and the owner can exempt individual callers from paying. Native payments forward only the required fee and refund excess value; ERC-20 payments use `safeTransferFrom` and reject attached native value. Owner updates are timelocked and replace the fee token together with the whole fee table atomically. New getters and events expose the configuration. Home deployments remain fee-free and reject fee configuration.
- **Caller-selected signature thresholds.** `Relay.verifyCustomSignatureWithThreshold()` and its FDC2 wrapper allow verification against a supplied fraction of total signing weight. Zero selects the policy's existing threshold; nonzero values require strictly more than the specified fraction, and values of 10000 BIPS or above are rejected. The override uses transient storage and applies only to custom-message verification, leaving policy relay and protocol finalization thresholds unaffected.

- **Relay validation.** `Relay.relay()` adds canonical ECDSA checks for `v` and low `s`, explicitly rejects zero recovered signers, rejects zero roots during finalization. `Relay.verify()` correspondingly requires a finalized, nonzero root on the local verification path. The `oldRelay` delegation path, however, carries no equivalent guard, see Finding [2.1](#), 7.
- **Payable Flare governance execution.** `executeGovernanceCall` becomes payable and forwards the executor's attached value into the timelocked call. Recording a governance call now rejects attached value, so funding is supplied at execution time instead of being retained during scheduling. Note that the value is not part of the timelocked commitment, see Finding [2.2](#), 7.
- **Signed feed values and individual timestamps.** `FtsoV2` adds `getCurrentFeed`, `getCurrentFeeds`, and their wei-denominated variants. These return signed values, with batch reads reporting a timestamp for each feed. Existing unsigned getters reject negative values, existing batch getters require matching timestamps, and batch reads reject empty inputs.
- **Feed-read payment accounting.** `FtsoV2` calculates and forwards only the required custom-feed and `FastUpdater` fees, checks them against the current call's payment, and tracks the remaining payment across mixed batches. It no longer uses its entire contract balance to pay for fast updates. All external feed-read paths burn the current call's excess value while preserving the pre-existing balance.

4.2. Minor changes

The following are minor changes made to the codebase.

- Relay replaces string-based revert reasons with custom errors declared in `IRelay`, including explicit selectors for assembly revert paths, and simplifies policy encoding and calldata reads.
- TEE set indexing changes from `EnumerableSet.at` to `EnumerableSet.pos` in `MachineManagerFacet` and `ExtensionGovernance`, in order to comply with OpenZeppelin's deprecation of the `at` functions, which are in response to Solidity adding it as a reserved keyword.